

Отчет к лабораторной работе №1

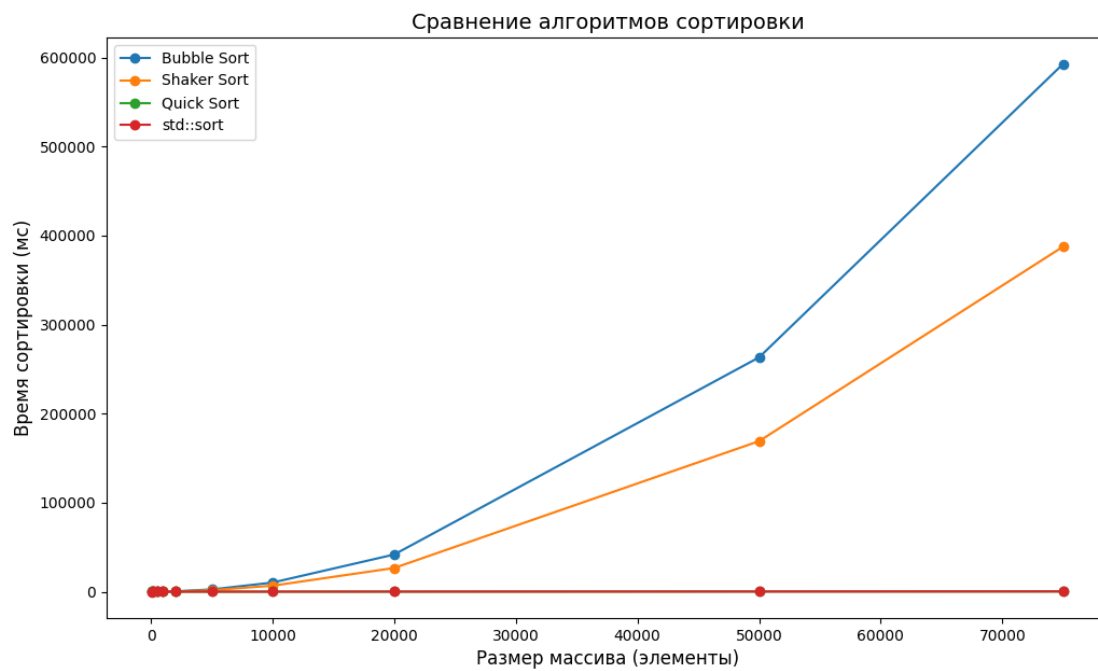
Вариант №21

Труфанова Анастасия СКБ222

Ссылка на исходный код программы в репозитории

https://github.com/neko-nyashka/lab1_sorting_algorithms/tree/develop/src

Графики времени сортировок



My Project

Generated by Doxygen 1.9.5

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Flight Class Reference	5
3.1.1 Detailed Description	5
3.1.2 Constructor & Destructor Documentation	5
3.1.2.1 Flight() [1/2]	5
3.1.2.2 Flight() [2/2]	6
3.1.3 Member Function Documentation	6
3.1.3.1 operator<()	6
3.1.3.2 operator<=()	6
3.1.3.3 operator==()	7
3.1.3.4 operator>()	7
3.1.3.5 operator>=()	7
3.1.4 Friends And Related Function Documentation	7
3.1.4.1 operator<<	7
4 File Documentation	9
4.1 flight.cpp	9
4.2 flight.h	10
4.3 src/sort.cpp File Reference	10
4.3.1 Detailed Description	10
4.3.2 Function Documentation	11
4.3.2.1 bubbleSort()	11
4.3.2.2 main()	11
4.3.2.3 parseCSV()	11
4.3.2.4 quickSort()	12
4.3.2.5 shakerSort()	12
4.4 sort.cpp	13
4.5 sort.h	15
Index	17

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Flight	5
----------------------------------	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/ flight.cpp	9
src/ flight.h	10
src/ sort.cpp	
Implementation of sorting algorithms and CSV parsing	10
src/ sort.h	15

Chapter 3

Class Documentation

3.1 Flight Class Reference

Public Member Functions

- [Flight](#) ()
- [Flight](#) (std::string flight_number_, std::string airline_, std::string arrival_date_, std::string arrival_time_, int passengers_)
- bool [operator>](#) (const [Flight](#) &other) const
- bool [operator<](#) (const [Flight](#) &other) const
- bool [operator<=](#) (const [Flight](#) &other) const
- bool [operator>=](#) (const [Flight](#) &other) const
- bool [operator==](#) (const [Flight](#) &other) const
- [Flight](#) & [operator=](#) (const [Flight](#) &other)=default

Friends

- std::ostream & [operator<<](#) (std::ostream &os, const [Flight](#) &flight)

3.1.1 Detailed Description

Definition at line 5 of file [flight.h](#).

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Flight() [1/2]

```
Flight::Flight ( )
```

Default constructor. Initializes all fields to empty/zero values.

Definition at line 6 of file [flight.cpp](#).

3.1.2.2 Flight() [2/2]

```
Flight::Flight (
    std::string flight_number_,
    std::string airline_,
    std::string arrival_date_,
    std::string arrival_time_,
    int passengers_ )
```

Parameterized constructor. Initializes fields with provided values.

Parameters

<i>flight_↔ number_</i>	Flight number (e.g., "SU123").
<i>airline_</i>	Airline name (e.g., "Aeroflot").
<i>arrival_date_↔ _</i>	Arrival date (e.g., "2023-10-15").
<i>arrival_time_↔ _</i>	Arrival time (e.g., "14:30").
<i>passengers_↔ _</i>	Passenger count (e.g., 150).

Definition at line 24 of file [flight.cpp](#).

3.1.3 Member Function Documentation

3.1.3.1 operator<()

```
bool Flight::operator< (
    const Flight & other ) const
```

Definition at line 52 of file [flight.cpp](#).

3.1.3.2 operator<=()

```
bool Flight::operator<= (
    const Flight & other ) const
```

Definition at line 56 of file [flight.cpp](#).

3.1.3.3 operator==()

```
bool Flight::operator==(   
    const Flight & other ) const
```

Definition at line 60 of file [flight.cpp](#).

3.1.3.4 operator>()

```
bool Flight::operator> (   
    const Flight & other ) const
```

Definition at line 32 of file [flight.cpp](#).

3.1.3.5 operator>=()

```
bool Flight::operator>= (   
    const Flight & other ) const
```

Definition at line 45 of file [flight.cpp](#).

3.1.4 Friends And Related Function Documentation

3.1.4.1 operator<<

```
std::ostream & operator<< (   
    std::ostream & os,   
    const Flight & flight ) [friend]
```

Definition at line 68 of file [flight.cpp](#).

The documentation for this class was generated from the following files:

- [src/flight.h](#)
- [src/flight.cpp](#)

Chapter 4

File Documentation

4.1 flight.cpp

```
00001 #include "flight.h"
00006 Flight::Flight() {
00007     flight_number = "";
00008     airline = "";
00009     arrival_date = "";
00010     arrival_time = "";
00011     passengers = 0;
00012 }
00013
00024 Flight::Flight(std::string flight_number_, std::string airline_, std::string arrival_date_,
std::string arrival_time_, int passengers_) {
00025     flight_number = flight_number_;
00026     airline = airline_;
00027     arrival_date = arrival_date_;
00028     arrival_time = arrival_time_;
00029     passengers = passengers_;
00030 }
00031
00032 bool Flight::operator>(const Flight& other) const {
00033     if (arrival_date != other.arrival_date) {
00034         return (arrival_date > other.arrival_date);
00035     }
00036     if (arrival_time != other.arrival_time) {
00037         return (arrival_time > other.arrival_time);
00038     }
00039     if (airline != other.airline) {
00040         return airline > other.airline;
00041     }
00042     return passengers > other.passengers;
00043 }
00044
00045 bool Flight::operator>=(const Flight& other) const {
00046     if (arrival_date != other.arrival_date) return arrival_date > other.arrival_date;
00047     if (arrival_time != other.arrival_time) return arrival_time > other.arrival_time;
00048     if (airline != other.airline) return airline > other.airline;
00049     return passengers <= other.passengers;
00050 }
00051
00052 bool Flight::operator<(const Flight& other) const {
00053     return !(*this >= other);
00054 }
00055
00056 bool Flight::operator<=(const Flight& other) const {
00057     return !(*this > other);
00058 }
00059
00060 bool Flight::operator==(const Flight& other) const {
00061     return arrival_date == other.arrival_date &&
00062         arrival_time == other.arrival_time &&
00063         airline == other.airline &&
00064         passengers == other.passengers;
00065 }
00066
00067
00068 std::ostream& operator<<(std::ostream& os, const Flight& flight) {
00069     os << flight.flight_number << " " << flight.airline << " " << flight.arrival_date << " " <<
flight.arrival_time << " " << flight.passengers;
00070     return os;
00071 }
00072
```


4.2 flight.h

```

00001 #ifndef FLIGHT_H
00002 #define FLIGHT_H
00003 #include <string>
00004 #include <iostream>
00005 class Flight
00006 {
00007 private:
00008     std::string flight_number;
00009     std::string airline;
00010     std::string arrival_date;
00011     std::string arrival_time;
00012     int passengers;
00013
00014 public:
00015     Flight();
00016     Flight(std::string flight_number_, std::string airline_, std::string arrival_date_, std::string
arrival_time_, int passengers_);
00017     ~Flight() = default;
00018     bool operator>(const Flight& other) const;
00019     bool operator<(const Flight& other) const;
00020     bool operator<=(const Flight& other) const;
00021     bool operator>=(const Flight& other) const;
00022     bool operator==(const Flight& other) const;
00023     Flight& operator=(const Flight& other) = default;
00024     friend std::ostream& operator<<(std::ostream& os, const Flight& flight);
00025 };
00026
00027 #endif

```

4.3 src/sort.cpp File Reference

Implementation of sorting algorithms and CSV parsing.

```
#include "sort.h"
```

Functions

- void [parseCSV](#) (std::string filename, int N, [Flight](#) *result)
- template<class T >
void [bubbleSort](#) (T *a, int N)
- template<class T >
void [shakerSort](#) (T *a, int N)
- template<class T >
void [quickSort](#) (T *a, long N)
- int [main](#) ()

4.3.1 Detailed Description

Implementation of sorting algorithms and CSV parsing.

Contains:

- [parseCSV\(\)](#) function for reading flight data from CSV files
- Template implementations of:
 - Bubble Sort
 - Shaker Sort (Cocktail Sort)
 - Quick Sort
- Main function with performance benchmarking

Definition in file [sort.cpp](#).

4.3.2 Function Documentation

4.3.2.1 bubbleSort()

```
template<class T >
void bubbleSort (
    T * a,
    int N )
```

Sorts an array in descending order using the Bubble Sort algorithm.

Template Parameters

<i>T</i>	The type of elements in the array.
----------	------------------------------------

Parameters

<i>a</i>	Pointer to the array to be sorted.
<i>N</i>	The number of elements in the array.

Definition at line 67 of file [sort.cpp](#).

4.3.2.2 main()

```
int main ( )
```

Definition at line 141 of file [sort.cpp](#).

4.3.2.3 parseCSV()

```
void parseCSV (
    std::string filename,
    int N,
    Flight * result )
```

Parses a CSV file containing flight data and stores the results in an array of [Flight](#) objects.

Parameters

<i>filename</i>	The path to the CSV file.
<i>N</i>	The maximum number of entries to read.
<i>result</i>	Pointer to the array where Flight objects will be stored.

Definition at line 21 of file [sort.cpp](#).

4.3.2.4 quickSort()

```
template<class T >
void quickSort (
    T * a,
    long N )
```

Sorts an array in descending order using the Quick Sort algorithm.

Template Parameters

<i>T</i>	The type of elements in the array.
----------	------------------------------------

Parameters

<i>a</i>	Pointer to the array to be sorted.
<i>N</i>	The number of elements in the array.

Definition at line 120 of file [sort.cpp](#).

4.3.2.5 shakerSort()

```
template<class T >
void shakerSort (
    T * a,
    int N )
```

Sorts an array in descending order using the Shaker Sort (Bidirectional Bubble Sort) algorithm.

Template Parameters

<i>T</i>	The type of elements in the array.
----------	------------------------------------

Parameters

<i>a</i>	Pointer to the array to be sorted.
<i>N</i>	The number of elements in the array.

Definition at line 85 of file [sort.cpp](#).

4.4 sort.cpp

[Go to the documentation of this file.](#)

```

00001 #include "sort.h"
00021 void parseCSV(std::string filename, int N, Flight* result) {
00022     std::ifstream in(filename);
00023     std::string line, flight_number_, airline_, arrival_date_, arrival_time_;
00024     int i1 = 0, i2 = 0, i3 = 0, i4 = 0, i = 0, passengers_;
00025
00026     if (in.is_open()) {
00027         while (std::getline(in, line) && i < N) {
00028             // Locate the positions of commas to separate CSV fields
00029             for (int i = 0; i < (int)line.size(); i++) {
00030                 if (line[i] == ',') {
00031                     if (i1 == 0) i1 = i;
00032                     else if (i2 == 0) i2 = i;
00033                     else if (i3 == 0) i3 = i;
00034                     else if (i4 == 0) i4 = i;
00035                 }
00036             }
00037
00038             // Extract fields using the comma positions
00039             flight_number_ = line.substr(0, i1);
00040             airline_ = line.substr(i1 + 1, i2 - i1 - 1);
00041             arrival_date_ = line.substr(i2 + 1, i3 - i2 - 1);
00042             arrival_time_ = line.substr(i3 + 1, i4 - i3 - 1);
00043             passengers_ = std::stoi(line.substr(i4 + 1, line.size() - i4 - 2));
00044
00045             // Reset comma position trackers for the next line
00046             i1 = 0;
00047             i2 = 0;
00048             i3 = 0;
00049             i4 = 0;
00050
00051             // Create a Flight object and store it in the result array
00052             Flight f(flight_number_, airline_, arrival_date_, arrival_time_, passengers_);
00053             result[i++] = f;
00054         }
00055         in.close();
00056     }
00057 }
00058
00066 template<class T>
00067 void bubbleSort(T* a, int N) {
00068     for (int i = 0; i < N; ++i) {
00069         for (int j = N - 1; j > i; --j) {
00070             if (a[j - 1] < a[j]) {
00071                 std::swap(a[j - 1], a[j]);
00072             }
00073         }
00074     }
00075 }
00076
00084 template<class T>
00085 void shakerSort(T* a, int N) {
00086     int lb = 0;           // Lower bound
00087     int ub = N - 1;       // Upper bound
00088     int k;                // Tracks the last swap position
00089
00090     do {
00091         k = 0;
00092         // Pass from right to left
00093         for (int j = ub; j > lb; --j) {
00094             if (a[j - 1] < a[j]) {
00095                 std::swap(a[j - 1], a[j]);
00096                 k = j;
00097             }
00098         }
00099         lb = k;
00100
00101         // Pass from left to right
00102         for (int j = lb; j < ub; ++j) {
00103             if (a[j] < a[j + 1]) {
00104                 std::swap(a[j], a[j + 1]);
00105                 k = j;
00106             }
00107         }
00108         ub = k;
00109     } while (lb < ub);
00110 }
00111
00119 template<class T>
00120 void quickSort(T* a, long N) {
00121     long i = 0, j = N - 1;    // Initialize bounds
00122     T p = a[N >> 1];         // Pivot: middle element

```

```

00123
00124 // Partitioning phase
00125 do {
00126     while (i < N && a[i] > p) i++;
00127     while (j >= 0 && a[j] < p) j--;
00128
00129     if (i <= j) {
00130         std::swap(a[i], a[j]);
00131         i++;
00132         j--;
00133     }
00134 } while (i <= j);
00135
00136 // Recursively sort the partitions
00137 if (j > 0) quickSort(a, j + 1);
00138 if (N > i) quickSort(a + i, N - i);
00139 }
00140
00141 int main() {
00142     std::ofstream out("/Users/anastasiatrufanova/Desktop/lab1_sorting_algorithms/plt/results.csv");
00143     out << "Size,BubbleSort,ShakerSort,QuickSort,StdSort\n";
00144
00145     std::vector<std::string> filenames = {
00146         "100.csv",
00147         "200.csv",
00148         "500.csv",
00149         "1000.csv",
00150         "2000.csv",
00151         "5000.csv",
00152         "10000.csv",
00153         "20000.csv",
00154         "50000.csv",
00155         "75000.csv"
00156     };
00157
00158     std::string path = "/Users/anastasiatrufanova/Desktop/lab1_sorting_algorithms/data/";
00159
00160     for (int i = 0; i < 10; ++i) {
00161         std::string full_path = path + filenames[i];
00162         int N = std::stoi(filenames[i].substr(0, filenames[i].size() - 4));
00163         Flight* original = new Flight[N];
00164         parseCSV(full_path, N, original);
00165
00166         // --- Bubble Sort ---
00167         Flight* a1 = new Flight[N];
00168         std::copy(original, original + N, a1);
00169         auto t1_start = std::chrono::high_resolution_clock::now();
00170         bubbleSort(a1, N);
00171         auto t1_end = std::chrono::high_resolution_clock::now();
00172         double t_bubble = std::chrono::duration<double, std::milli>(t1_end - t1_start).count();
00173         delete[] a1;
00174
00175         // --- Shaker Sort ---
00176         Flight* a2 = new Flight[N];
00177         std::copy(original, original + N, a2);
00178         auto t2_start = std::chrono::high_resolution_clock::now();
00179         shakerSort(a2, N);
00180         auto t2_end = std::chrono::high_resolution_clock::now();
00181         double t_shaker = std::chrono::duration<double, std::milli>(t2_end - t2_start).count();
00182         delete[] a2;
00183
00184         // --- Quick Sort ---
00185         Flight* a3 = new Flight[N];
00186         std::copy(original, original + N, a3);
00187         auto t3_start = std::chrono::high_resolution_clock::now();
00188         quickSort(a3, N);
00189         auto t3_end = std::chrono::high_resolution_clock::now();
00190         double t_quick = std::chrono::duration<double, std::milli>(t3_end - t3_start).count();
00191         delete[] a3;
00192
00193         // --- std::sort (for comparison) ---
00194         Flight* a4 = new Flight[N];
00195         std::copy(original, original + N, a4);
00196         auto t4_start = std::chrono::high_resolution_clock::now();
00197         std::sort(a4, a4 + N, [](const Flight& a, const Flight& b) {
00198             return a > b; // Sort in descending order
00199         });
00200         auto t4_end = std::chrono::high_resolution_clock::now();
00201         double t_std = std::chrono::duration<double, std::milli>(t4_end - t4_start).count();
00202         delete[] a4;
00203
00204         delete[] original;
00205
00206         // Write results to CSV
00207         out << N << "," << t_bubble << "," << t_shaker << "," << t_quick << "," << t_std << "\n";
00208     }
00209

```

```
00210     out.close();
00211     return 0;
00212 }
```

4.5 sort.h

```
00001 #ifndef SORT_H
00002 #define SORT_H
00003 #include <string>
00004 #include "flight.h"
00005 #include <fstream>
00006 #include <algorithm> // std::sort
00007 #include <chrono>
00008 void parseCSV(std::string filename, int N, Flight *result);
00009 template<class T>
00010 void bubbleSort(T* a, int N);
00011 template<class T>
00012 void shakerSort(T* a, int N);
00013 template<class T>
00014 void quickSort(T *a, long N);
00015 #endif
```


Index

- `bubbleSort`
 - `sort.cpp`, [11](#)
- `Flight`, [5](#)
 - `Flight`, [5](#)
 - `operator<`, [6](#)
 - `operator<<`, [7](#)
 - `operator<=`, [6](#)
 - `operator>`, [7](#)
 - `operator>=`, [7](#)
 - `operator==`, [6](#)
- `main`
 - `sort.cpp`, [11](#)
- `operator<`
 - `Flight`, [6](#)
- `operator<<`
 - `Flight`, [7](#)
- `operator<=`
 - `Flight`, [6](#)
- `operator>`
 - `Flight`, [7](#)
- `operator>=`
 - `Flight`, [7](#)
- `operator==`
 - `Flight`, [6](#)
- `parseCSV`
 - `sort.cpp`, [11](#)
- `quickSort`
 - `sort.cpp`, [12](#)
- `shakerSort`
 - `sort.cpp`, [12](#)
- `sort.cpp`
 - `bubbleSort`, [11](#)
 - `main`, [11](#)
 - `parseCSV`, [11](#)
 - `quickSort`, [12](#)
 - `shakerSort`, [12](#)
- `src/flight.cpp`, [9](#)
- `src/flight.h`, [10](#)
- `src/sort.cpp`, [10](#), [13](#)
- `src/sort.h`, [15](#)